WARSZTAT PRAKTYCZNY

Typ VECTOR w SQL Server 2025:
wyszukiwanie podobnych zgłoszeń w helpdesku

Realny scenariusz dla działu obsługi klienta sklepu internetowego „MiętaMarket”. Od pustej bazy, przez typ `vector` i funkcję `VECTOR_DISTANCE`, automatyczne podpowiadanie odpowiedzi i audyt jakości danych, aż po indeks DiskANN i wyszukiwanie przybliżone `VECTOR_SEARCH`.

Parametr	Wartość
Szkolenie	Użycie AI w pracy z SQL Server
Grupa docelowa	Zaawansowani użytkownicy SQL / T-SQL; wiedza o embeddingach niewymagana
Środowisko	SQL Server 2025 (17.x) Developer Edition, SSMS 21 lub nowszy
Baza	`WektoryHelpdesk` – tworzona od zera skryptem z Modułu 1 (dane fikcyjne, deterministyczne)
Struktura	4 moduły; każdy kończy się skryptem weryfikacyjnym ze statusem OK / BŁĄD
Funkcje preview	Moduł 4 (indeks wektorowy, `VECTOR_SEARCH`) wymaga `PREVIEW_FEATURES = ON`; typ `vector` i `VECTOR_DISTANCE` są GA

Spis treści

Spis treści 2
0. Informacje 3
Cele dydaktyczne 3
Scenariusz biznesowy 3
Embedding w jednym akapicie 3
Model danych warsztatu 4
Moduł 1. Typ VECTOR i dane od zera 5
Ćwiczenie 1.1 – utworzenie bazy i wygenerowanie danych 5
Ćwiczenie 1.2 – typ VECTOR w pigułce 5
Weryfikacja Modułu 1 6
Moduł 2. Podobne zgłoszenia i automatyczna odpowiedź 7
Ćwiczenie 2.1 – pięć najbardziej podobnych zgłoszeń 7
Ćwiczenie 2.2 – propozycja tematu dla wszystkich nowych zgłoszeń 7
Ćwiczenie 2.3 – czy metryka ma znaczenie? 8
Weryfikacja Modułu 2 8
Moduł 3. Centroidy i audyt błędnych etykiet 9
Ćwiczenie 3.1 – centroidy kategorii 9
Ćwiczenie 3.2 – które zgłoszenia mają podejrzaną kategorię? 9
Ćwiczenie 3.3 – korekta etykiet po akceptacji 9
Weryfikacja Modułu 3 10
Moduł 4. Indeks DiskANN i VECTOR_SEARCH 11
Wprowadzenie 11
Ćwiczenie 4.1 – utworzenie indeksu 11
Ćwiczenie 4.2 – VECTOR_SEARCH 11
Ćwiczenie 4.3 – pomiar recall@10 11
Weryfikacja Modułu 4 12
Podsumowanie 13
Co zbudowaliśmy 13
Pytania do dyskusji 13
Zadania dodatkowe dla chętnych 13
Pytania kontrolne (z odpowiedziami) 13
Co przygotować przed zajęciami 14
Załącznik A (opcjonalny). Prawdziwe embeddingi z modelu 14
Załącznik B. Sprzątanie po warsztacie 14
Źródła 14

0. Informacje

Cele dydaktyczne

Po warsztacie:

- tworzy i konwertuje dane typu `vector`, odczytuje ich właściwości (`VECTORPROPERTY`) i normalizuje je (`VECTOR_NORM`, `VECTOR_NORMALIZE`),
- rozumie różnicę między metrykami `cosine`, `euclidean` i `dot` oraz między odległością a podobieństwem,
- buduje wyszukiwanie dokładne (kNN) z `VECTOR_DISTANCE` i na jego podstawie automatycznie proponuje odpowiedź z bazy wiedzy, z progiem eskalacji do człowieka,
- wykorzystuje wektory do analityki: liczy centroidy kategorii i wykrywa błędnie sklasyfikowane rekordy,
- tworzy indeks DiskANN, używa `VECTOR_SEARCH` i mierzy jakość wyszukiwania przybliżonego (recall@10),
- krytycznie ocenia kod „wektorowy" generowany przez AI, który często miesza składnię SQL Server z pgvector i innymi bazami.

Scenariusz biznesowy

Dział obsługi klienta sklepu „MiętaMarket" dostaje kilkaset zgłoszeń miesięcznie przez e-mail, czat i telefon. Konsultanci za każdym razem szukają w archiwum podobnej sprawy i kopiują standardową odpowiedź z bazy wiedzy. Kierowniczka zespołu chce, żeby system sam podpowiadał **podobne zgłoszenia i gotowe rozwiązanie**, a sprawy nietypowe kierował do człowieka.

W bazie jest archiwum 1200 zgłoszeń z 2026 r. przypisanych do 24 tematów w 8 kategoriach oraz 10 nowych zgłoszeń z dzisiejszego poranka. Każde zgłoszenie ma **embedding** – 32-wymiarowy wektor opisujący znaczenie treści. Przy okazji wyjdzie na jaw problem znany z każdego helpdesku: część archiwalnych zgłoszeń konsultanci przypisali do złej kategorii.

Embedding w jednym akapicie

Embedding to lista liczb, którą model językowy przypisuje tekstowi tak, aby teksty o podobnym znaczeniu dostawały wektory leżące blisko siebie. „Nie pamiętam hasła" i „link do resetu nie przychodzi" mają różne słowa, ale bliskie wektory. Bazie danych wystarczy więc policzyć odległość między wektorami, żeby znaleźć sprawy podobne znaczeniowo – bez dopasowywania słów kluczowych.

Metryka w VECTOR_DISTANCE	Co liczy	Interpretacja
`'cosine'`	1 − cos(kąt między wektorami)	0 = ten sam kierunek, 1 = prostopadłe, 2 = przeciwne. Podobieństwo = 1 − odległość.
`'euclidean'`	Długość odcinka między punktami	Wrażliwa na długość wektora; dla wektorów znormalizowanych daje tę samą kolejność co cosine.
`'dot'`	Ujemny iloczyn skalarny	Im mniejsza (bardziej ujemna), tym bliżej. Dla wektorów o długości 1: dot = cosine − 1.

Ważne – skąd biorą się wektory? W produkcji embedding wylicza zewnętrzny model (np. przez `AI_GENERATE_EMBEDDINGS` – Załącznik A). Na warsztacie wektory są **symulowane** deterministycznie: wektor = składowa kategorii + składowa tematu + szum, następnie normalizacja. Dzięki temu warsztat działa bez dostępu do sieci i modeli, a każdy uczestnik ma identyczne dane i te same wartości w weryfikacjach. Powiedz to grupie wprost – mechanika SQL jest dokładnie taka sama jak dla prawdziwych embeddingów (zwykle 384–1536 wymiarów).

Model danych warsztatu

Tabela	Zawartość	Tworzona w
`dbo.Kategoria`, `dbo.Temat`	8 kategorii, 24 tematy ze standardowym rozwiązaniem z bazy wiedzy	Moduł 1
`dbo.Zgloszenie`	1200 zgłoszeń: data, kanał, temat przypisany przez konsultanta, treść, `Embedding vector(32)`	Moduł 1
`dbo.NoweZgloszenie`	10 dzisiejszych zgłoszeń czekających na odpowiedź (jedno celowo niejednoznaczne)	Moduł 1
`dbo.KluczWeryfikacji`	Oczekiwane tematy nowych zgłoszeń – używa go tylko skrypt weryfikacyjny	Moduł 1
`dbo.Propozycja`	Proponowany temat, liczba głosów, podobieństwo, decyzja (auto / konsultant)	Moduł 2
`dbo.CentroidKategorii`, `dbo.PodejrzaneZgloszenie`	Średnie wektory kategorii i wynik audytu etykiet	Moduł 3
`dbo.WynikANN`, `dbo.WynikDokladny`	Wyniki wyszukiwania przybliżonego i dokładnego do pomiaru recall	Moduł 4

Kolejność modułów ma znaczenie. Moduł 3 poprawia etykiety w `dbo.Zgloszenie`, a Moduł 4 tworzy indeks wektorowy. W starszych wersjach indeksu (preview) tabela z indeksem DiskANN staje się tylko do odczytu, dlatego indeks powstaje dopiero na końcu.

Moduł 1. Typ VECTOR i dane od zera

Cel modułu
Utworzyć bazę WektoryHelpdesk z kompletem danych, poznać typ vector, konwersje, metryki odległości i normalizację.

Ćwiczenie 1.1 – utworzenie bazy i wygenerowanie danych

Uruchom skrypt w całości (F5). Usuwa on bazę `WektoryHelpdesk`, jeśli istnieje, więc warsztat można w każdej chwili zacząć od nowa. Wykonanie trwa kilka sekund.

Skrypt: m1_setup.sql

Na co zwrócić uwagę:

- Wektor powstaje z tekstu JSON (`[0.12,-0.5,…]`) przez `CAST(… AS vector(32))`. To samo robi aplikacja, która dostaje embedding z API modelu.
- `VECTOR_NORMALIZE(…, 'norm2')` sprowadza każdy wektor do długości 1 – większość modeli embeddingów zwraca wektory już znormalizowane.
- Kolumna `TematID` to etykieta nadana przez konsultanta. Dla 12 zgłoszeń jest celowo błędna – treść mówi o czymś innym. Odkryjemy to w Module 3.
- Zgłoszenie nowe nr 10 dotyczy dwóch spraw naraz (uszkodzenie i brak zwrotu pieniędzy) – to test, czy system rozpozna, że powinien oddać sprawę człowiekowi.

Ćwiczenie 1.2 – typ VECTOR w pigułce

Na trzech wymiarach łatwo policzyć wyniki w pamięci i sprawdzić, czy funkcje działają zgodnie z intuicją. Uruchamiaj kolejne bloki osobno.

Skrypt: m1_basics.sql

Oczekiwane wyniki (blok b):

Para	Cosine	Euclidean	Dot
a vs b	0,2	0,632	-0,8
a vs c	1	2,236	0

Wartości mogą mieć „ogon” w rodzaju 0,19999998 – elementy wektora są przechowywane jako float32 (4 bajty). Tekst z bloku a) to tablica JSON w notacji wykładniczej. W bloku c) długość `@c` to 2, a po normalizacji wektor `[0, 0, 1]` ma długość 1. **Pytanie do grupy:** dlaczego a vs c ma odległość kosinusową 1, mimo że euklidesowa jest „duża” tylko z powodu długości c? (Cosine patrzy wyłącznie na kierunek.)

Informacje: gdy kod pisze asystent AI

Przykładowy prompt: „Jak w SQL Server zapisać embedding i znaleźć 5 najbardziej podobnych rekordów?”

- Modele często podają składnię **pgvector** z PostgreSQL: operatory `<=>`, `<->`, `<#>`, typ `vector` bez nawiasów, `vector_cosine_ops`. W SQL Server nie ma tych operatorów – jest funkcja `VECTOR_DISTANCE`.
- Starsze odpowiedzi proponują przechowywanie embeddingów w `nvarchar(max)` jako JSON albo w `varbinary` i własną funkcję kosinusową w T-SQL lub CLR – przed SQL Server 2025 tak się robiło, dziś to zbędne i wolne.
- Mylenie odległości z podobieństwem: `VECTOR_DISTANCE('cosine', …)` zwraca odległość (mniej = bliżej), więc sortujemy rosnąco.
- Nieistniejące nazwy metryk, np. `'cosine_similarity'` czy `'l2'`. Poprawne: `'cosine'`, `'euclidean'`, `'dot'`.

- Bezpieczeństwo: embedding z danych osobowych nadal jest daną osobową – podlega RODO i nie powinien trafiać do zewnętrznych usług bez podstawy prawnej.

Weryfikacja Modułu 1

Skrypt: m1_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 12 testów powinno mieć status **OK**, a podsumowanie: „Moduł 1 zaliczony – możesz przejść dalej”.

- Test 1 = BŁĄD → instancja starsza niż SQL Server 2025; typ `vector` nie jest dostępny.
- Błąd „Cannot find data type vector” → sprawdź wersję serwera i poziom zgodności bazy (`SELECT compatibility_level FROM sys.databases WHERE name = DB_NAME();`).
- Testy 2–8 = BŁĄD → skrypt 1.1 nie wykonał się do końca; uruchom go ponownie w całości.

Moduł 2. Podobne zgłoszenia i automatyczna odpowiedź

Cel modułu
Zbudować wyszukiwanie dokładne (kNN), na jego podstawie proponować temat i odpowiedź z bazy wiedzy, a sprawy niepewne kierować do konsultanta.

Ćwiczenie 2.1 – pięć najbardziej podobnych zgłoszeń

Dokładne wyszukiwanie najbliższych sąsiadów (kNN, k-nearest neighbors) to zwykłe `TOP (k) … ORDER BY VECTOR_DISTANCE(…)`. Silnik liczy odległość do każdego wiersza – przy 1200 zgłoszeniach to ułamek sekundy, a dokumentacja Microsoft wskazuje, że wyszukiwanie dokładne jest rozsądne do ok. 50 tys. wektorów.

Skrypt: m2_ex1.sql

Oczekiwany wynik: wszystkie 5 zgłoszeń ma temat „Reset zapomnianego hasła”, a podobieństwo wynosi ok. 0,94–0,95:

ZgloszenieID	Tytul	Podobienstwo
198	Reset zapomnianego hasła	0,946
534	Reset zapomnianego hasła	0,945
417	Reset zapomnianego hasła	0,944
1180	Reset zapomnianego hasła	0,944
992	Reset zapomnianego hasła	0,943

Ćwiczenie 2.2 – propozycja tematu dla wszystkich nowych zgłoszeń

Reguła biznesowa: 5 najbliższych sąsiadów głosuje na temat. Odpowiedź wysyłamy automatycznie tylko wtedy, gdy najbliższy sąsiad jest naprawdę podobny (≥ 0,85) i co najmniej 3 z 5 sąsiadów wskazują ten sam temat. W pozostałych przypadkach sprawa trafia do konsultanta.

Skrypt: m2_ex2.sql

Oczekiwany wynik (wartości podobieństwa orientacyjne):

NoweID	ProponowanyTemat	Glosy	MaksPodob.	Decyzja
1	Reset zapomnianego hasła	5	0,946	Automatyczna odpowiedź
2	Podwójne obciążenie karty	5	0,973	Automatyczna odpowiedź
3	Opóźniona przesyłka	4	0,958	Automatyczna odpowiedź
4	Brak zwrotu pieniędzy	5	0,947	Automatyczna odpowiedź
5	Błędne dane na fakturze	5	0,965	Automatyczna odpowiedź
6	Aplikacja się zawiesza	5	0,975	Automatyczna odpowiedź
7	Kod rabatowy nie działa	5	0,943	Automatyczna odpowiedź
8	Wada fabryczna – reklamacja	5	0,965	Automatyczna odpowiedź
9	Nieudane doręczenie lub pełny paczkomat	5	0,962	Automatyczna odpowiedź
10	Brak zwrotu pieniędzy	5	0,682	Do konsultanta

Dyskusja: zgłoszenie 10 mówi o dwóch sprawach, więc jego wektor leży „pomiędzy” kategoriami i nawet najbliższy sąsiad jest mało podobny. Głosowanie byłoby jednomyślne, ale próg podobieństwa ratuje sytuację. Skąd wziąć próg

0,85? Z danych historycznych – w tym zbiorze typowe podobieństwo zgłoszeń z tego samego tematu to ok. 0,92, z tej samej kategorii ok. 0,69, a z różnych kategorii ok. 0.

Ćwiczenie 2.3 – czy metryka ma znaczenie?

Skrypt: m2_ex3.sql

Oczekiwany wynik: kolumny `R_Cosine`, `R_Euclidean` i `R_Dot` mają identyczne pozycje – dla wektorów znormalizowanych wszystkie trzy metryki dają tę samą kolejność. Dlatego `dot` (najtańszy obliczeniowo) jest popularny przy znormalizowanych embeddingach. **Zwróć uwagę grupy na pierwszy wiersz:** najbliższe zgłoszenie (ID 1101) ma przypisany temat „Aplikacja się zawiesza”, choć jego treść mówi o spóźnionej paczce. To błędna etykieta – głosowanie 5 sąsiadów w ćw. 2.2 ją „przegłosowało” (4 głosy na „Opóźnioną przesyłkę”). Tym zajmiemy się w Module 3.

Informacje: gdy kod pisze asystent AI

Przykładowy prompt: „Napisz zapytanie T-SQL, które dla każdego nowego zgłoszenia znajdzie najbardziej podobne zgłoszenie z archiwum”.

- Sortowanie malejąco po `VECTOR_DISTANCE` (pomylenie z podobieństwem) – zapytanie zwraca najmniej podobne rekordy, a wynik „wygląda” wiarygodnie.
- Brak `TOP` w `CROSS APPLY` albo `TOP` bez `ORDER BY` – przypadkowy wiersz zamiast najbliższego.
- Decyzja na podstawie jednego, najbliższego sąsiada: jak pokazało zgłoszenie 3, wystarczy jedna błędna etykieta w archiwum, by odpowiedź była zła.
- „Magiczny” próg podobieństwa (np. 0,8 albo 0,5) bez sprawdzenia na danych – progi zależą od modelu embeddingów i trzeba je kalibrować.
- Przy metryce `dot` model zapomina, że SQL Server zwraca ujemny iloczyn skalarny, i odwraca kolejność.

Weryfikacja Modułu 2

Skrypt: m2_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 8 testów powinno mieć status **OK**.

- Test 3 = BŁĄD → sprawdź kierunek sortowania w `CROSS APPLY` (rosnąco po odległości) i regułę głosowania.
- Test 5 lub 6 = BŁĄD → sprawdź, czy podobieństwo liczysz jako `1 - VECTOR_DISTANCE('cosine', …)`, a nie jako samą odległość.
- Test 8 = BŁĄD po wykonaniu Modułu 3 jest oczekiwany (etykieta zgłoszenia 1101 została już poprawiona). Weryfikację Modułu 2 uruchamiaj przed Modułem 3.

Moduł 3. Centroidy i audyt błędnych etykiet

Cel modułu
Policzyć średni wektor (centroid) każdej kategorii, znaleźć zgłoszenia leżące bliżej obcej kategorii i po akceptacji człowieka poprawić ich etykiety.

Ćwiczenie 3.1 – centroidy kategorii

SQL Server nie ma agregatu `AVG` dla typu `vector`. Rozbijamy więc wektor na składowe (`OPENJSON`), uśredniamy każdy wymiar i składamy wektor z powrotem (`STRING_AGG` + `CAST`). Centroid normalizujemy, bo średnia wektorów o długości 1 jest krótsza niż 1.

Skrypt: m3_ex1.sql

Oczekiwany wynik – najbliższe pary kategorii (orientacyjnie): Dostawa ↔ Uszkodzony produkt ok. 0,60, Konto i logowanie → Uszkodzony produkt ok. 0,66, Zwroty ↔ Faktury ok. 0,69; najdalej od innych leży kategoria Płatności (ok. 0,87). Pary blisko siebie to miejsca, w których konsultanci najłatwiej się mylą – i gdzie warto doprecyzować instrukcje.

Ćwiczenie 3.2 – które zgłoszenia mają podejrzaną kategorię?

Skrypt: m3_ex2.sql

Oczekiwany wynik: dokładnie 12 zgłoszeń. Na górze raportu (największa różnica odległości) m.in.:

ZgloszenieID	Przypisana kategoria	Proponowana kategoria	OdlPrzyp.	OdlProp.
222	Płatności	Program lojalnościowy	1,215	0,157
456	Uszkodzony produkt	Faktury	1,118	0,085
777	Zwroty	Konto i logowanie	1,072	0,050
37	Uszkodzony produkt	Faktury	1,116	0,102
…	…	…	…	…
388	Dostawa	Uszkodzony produkt	0,689	0,155

Przeczytajcie wspólnie treść kilku zgłoszeń: np. zgłoszenie 222 („punkty lojalnościowe zniknęły”) przypisano do Płatności. Wektor tylko wskazuje kandydatów – decyzję podejmuje człowiek.

Ćwiczenie 3.3 – korekta etykiet po akceptacji

Zakładamy, że kierowniczka zaakceptowała raport. Nowy temat ustalamy przez głosowanie 5 najbliższych zgłoszeń z proponowanej kategorii. Klauzula `OUTPUT` zostawia ślad zmian – w produkcji zapisz go do tabeli audytu.

Skrypt: m3_ex3.sql

Oczekiwany wynik: 12 wierszy, np. 37: 22 → 13, 141: 16 → 7, 222: 6 → 21.

Informacje: gdy kod pisze asystent AI

Przykładowy prompt: „Policz średni embedding dla każdej kategorii w SQL Server”.

- Najczęstsza odpowiedź to `AVG(Embedding)` – nie zadziała, agregaty nie obsługują typu `vector`.
- Brak normalizacji centroidu: przy metryce `cosine` to nie przeszkadza, ale przy `dot` i `euclidean` zmienia wyniki.

- Sortowanie składowych po `[key]` jako tekście (`'10' < '2'`) – wektor składa się w złej kolejności. Rzutuj klucz na `int`.
- Automatyczny `UPDATE` wszystkich „podejrzanych” bez przeglądu – wektor wskazuje kandydatów, a nie wyrok. W prawdziwych danych część „podejrzanych” to zgłoszenia nietypowe, a nie błędy.

Weryfikacja Modułu 3

Skrypt: m3_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 7 testów powinno mieć status **OK**.

- Test 4 ≠ 12 → najczęściej centroid policzono bez normalizacji lub z błędną kolejnością wymiarów.
- Testy 6–7 = BŁĄD → nie wykonano ćwiczenia 3.3 albo wykonano je przed 3.2 (pusta tabela podejrzanych).
- Po ponownym uruchomieniu ćw. 3.1–3.2 (już po korekcie) lista podejrzanych będzie pusta, a testy 4–5 zgłoszą BŁĄD – to oczekiwane. Aby powtórzyć moduł, uruchom ponownie skrypt 1.1 i ćwiczenia od początku.

Moduł 4. Indeks DiskANN i VECTOR_SEARCH

Cel modułu
Utworzyć indeks wektorowy, użyć wyszukiwania przybliżonego (ANN) i zmierzyć, ile wyników tracimy w zamian za szybkość.

Wprowadzenie

- **kNN (dokładne)** porównuje zapytanie z każdym wierszem – wynik idealny, koszt rośnie liniowo z liczbą wektorów.
- **ANN (przybliżone)** korzysta z grafu DiskANN i odwiedza tylko część wektorów – jest wielokrotnie szybsze przy milionach wierszy, ale może pominąć część prawdziwych sąsiadów. Miarą jakości jest **recall@k**: jaki odsetek prawdziwych k najbliższych zwrócił indeks.
- W SQL Server 2025 indeks wektorowy i `VECTOR_SEARCH` są w **wersji preview** – składnia zmieniała się między aktualizacjami. Starsza wersja używa parametru `TOP_N` i blokuje modyfikacje tabeli; nowsza używa `SELECT TOP (N) WITH APPROXIMATE` i obsługuje DML. Dlatego ćwiczenia mają dwa warianty.

Ćwiczenie 4.1 – utworzenie indeksu

Skrypt: m4_ex1.sql

Oczekiwany wynik: jeden wiersz – `IX_Zgloszenie_Embedding`, typ DiskANN, metryka cosine. Kolumna `WersjaIndeksu` pokazuje wersję (dokumentacja podaje, że 3 oznacza najnowszą – tutaj może być null). Indeks wymaga co najmniej 100 wierszy z niepustym wektorem (inaczej Msg 42266).

Ćwiczenie 4.2 – VECTOR_SEARCH

Skrypt: m4_ex2.sql

Oczekiwany wynik: 5 zgłoszeń o nieudanym doręczeniu lub pełnym paczkomacie. Kolumna `distance` jest dodawana przez `VECTOR_SEARCH` – jeśli tabela źródłowa miałaby własną kolumnę o tej nazwie, trzeba posłużyć się aliasami. Metryka w zapytaniu musi być taka sama jak w indeksie; w przeciwnym razie serwer zgłosi ostrzeżenie i wykona zwykłe kNN.

Ćwiczenie 4.3 – pomiar recall@10

Skrypt: m4_ex3.sql

Oczekiwany wynik: przy 1200 wektorach recall@10 wynosi zwykle 1,00 dla każdego zgłoszenia (optymalizator może zresztą w ogóle wybrać skan dokładny). Różnice pojawiają się przy setkach tysięcy wektorów – i właśnie wtedy warto tę procedurę pomiaru powtarzać po każdej zmianie danych lub parametrów.

Informacje: gdy kod pisze asystent AI

Przykładowy prompt: „Jak przyspieszyć wyszukiwanie wektorowe w SQL Server 2025?”

- Składnia z innych baz: `CREATE INDEX … USING hnsw (embedding vector_cosine_ops)` (pgvector) lub `ivfflat` – SQL Server ma `CREATE VECTOR INDEX … WITH (METRIC, TYPE = 'DiskANN')`.
- Pominięcie `ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON` – w SQL Server 2025 funkcja jest w preview.
- Mieszanie składni `TOP_N` i `TOP … WITH APPROXIMATE` w jednym zapytaniu; nowsze wersje zwracają przy `TOP_N` błąd Msg 42274.
- Zapytanie z filtrem `WHERE` przy starszej wersji indeksu: filtr działa po wyszukaniu, więc wynik może mieć mniej wierszy niż oczekiwano – model tego nie wspomni.

- Założenie, że ANN zwraca to samo co kNN. Zawsze mierz recall na własnych danych, zanim wdrożysz indeks.

Weryfikacja Modułu 4

Skrypt: m4_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 7 testów powinno mieć status **OK**, a podsumowanie: „Moduł 4 zaliczony – warsztat ukończony!”.

- Test 1 = BŁĄD → brak uprawnień do `ALTER DATABASE SCOPED CONFIGURATION` lub instrukcja nie została wykonana.
- Test 2 = BŁĄD → indeks nie powstał; przeczytaj komunikat z ćw. 4.1 (np. Msg 42266 – za mało wierszy).
- Testy 4–5 = BŁĄD → pętla w ćw. 4.3 przerwała się błędem składni; użyj drugiego wariantu `VECTOR_SEARCH`.

Podsumowanie

Co zbudowaliśmy

- Wyszukiwarkę znaczeniową zgłoszeń działającą w czystym T-SQL – bez kopiowania danych do zewnętrznej bazy wektorowej.
- Mechanizm automatycznej odpowiedzi z bezpiecznikiem (próg podobieństwa + głosowanie) i eskalacją do człowieka.
- Audyt jakości danych, który znalazł 12 błędnie przypisanych zgłoszeń wśród 1200.
- Indeks wektorowy ANN z pomiarem jakości, gotowy na wzrost danych.

Pytania do dyskusji

- Kiedy wystarczy kNN z `VECTOR_DISTANCE`, a kiedy potrzebny jest indeks? (Wskazówka: dokumentacja mówi o ok. 50 tys. wektorów po zastosowaniu filtrów.)
- Kto zatwierdza automatyczną odpowiedź wysyłaną klientowi i jak mierzyć, czy próg 0,85 jest właściwy?
- Jakie dane wolno zamienić na embeddingi u zewnętrznego dostawcy modelu, a jakie muszą zostać lokalnie?

Zadania dodatkowe dla chętnych

- Wygeneruj prawdziwe embeddingi z lokalnego modelu (Załącznik A) i porównaj wyniki ćwiczenia 2.2 z wersją symulowaną.
- Połącz wyszukiwanie wektorowe z pełnotekstowym (hybrid search): najpierw filtr `CONTAINS`, potem ranking po `VECTOR_DISTANCE`.
- Zbuduj prosty RAG: 3 najbliższe zgłoszenia i ich rozwiązania przekaż jako kontekst do modelu językowego, który zredaguje odpowiedź dla klienta.
- Sprawdź, jak zmienia się recall@10 po zwiększeniu tabeli do 100 tys. wierszy (użyj generatora z Modułu 1 z większym zakresem `GENERATE_SERIES`).

Pytania kontrolne (z odpowiedziami)

Pytanie	Odpowiedź
Jak utworzyć wektor z tekstu i z powrotem?	`CAST('[1, 2, 3]' AS vector(3))`; odwrotnie `CAST(@v AS nvarchar(max))` – wynikiem jest tablica JSON.
Dlaczego sortujemy rosnąco po `VECTOR_DISTANCE`?	Funkcja zwraca odległość: im mniejsza, tym bardziej podobne wektory. Podobieństwo kosinusowe = 1 − odległość.
Kiedy cosine, euclidean i dot dają tę samą kolejność?	Gdy wektory są znormalizowane do długości 1.
Jak policzyć średni wektor, skoro `AVG` nie działa na typie vector?	Rozbić wektor przez `OPENJSON`, uśrednić każdy wymiar, złożyć przez `STRING_AGG … WITHIN GROUP (ORDER BY wymiar)` i rzutować na vector.
Czym różni się kNN od ANN i jak ocenić ANN?	kNN jest dokładne i liczy odległość do wszystkich wierszy; ANN korzysta z indeksu i jest przybliżone. Jakość mierzy recall@k względem kNN.

Co przygotować przed zajęciami

- Instancja **SQL Server 2025 (17.x)** – sprawdź `SELECT @@VERSION;` i zanotuj numer aktualizacji CU; od niego zależy, który wariant składni `VECTOR_SEARCH` zadziała (przetestuj oba wcześniej).
- SSMS 21 lub nowszy – czytelnie wyświetla wartości typu `vector`.
- Uprawnienia uczestników: `CREATE DATABASE` (albo baza przygotowana wcześniej), `ALTER ANY DATABASE SCOPED CONFIGURATION` (Moduł 4) oraz `ALTER` na tabeli `dbo.Zgloszenie` (indeks).
- Przejdź cały warsztat raz na docelowej instancji: jeśli wszystkie cztery weryfikacje dadzą OK, warsztat zadziała.
- Skrypty z dokumentu zapisz jako pliki .sql – szybsze otwieranie i brak ryzyka zniekształceń przy kopiowaniu.
- Brak wymagań sieciowych: embeddingi są symulowane, a model zewnętrzny jest potrzebny tylko w opcjonalnym Załączniku A.
-

Załącznik A (opcjonalny). Prawdziwe embeddingi z modelu

SQL Server 2025 potrafi sam wywołać model embeddingów przez obiekt `EXTERNAL MODEL` i funkcję `AI_GENERATE_EMBEDDINGS`. Poniższy przykład dotyczy lokalnego serwera Ollama, więc dane nie opuszczają firmowej sieci. **Przed użyciem sprawdź aktualną dokumentację Microsoft Learn** – funkcje AI w SQL Server 2025 szybko się zmieniają, a część opcji jest w preview.

```
-- 1. Wywołania REST z silnika (sprawdź w dokumentacji, czy wymagane w Twojej wersji)
EXEC sp_configure 'external rest endpoint enabled', 1;
RECONFIGURE WITH OVERRIDE;
GO
-- 2. Model embeddingów na lokalnym Ollama (wymagany adres HTTPS)
CREATE EXTERNAL MODEL OllamaEmbeddingi
WITH (
    LOCATION   = 'https://localhost:11435/api/embed',
    API_FORMAT = 'Ollama',
    MODEL_TYPE = EMBEDDINGS,
    MODEL      = 'all-minilm'
);
GO
-- 3. Embedding dla nowego zgłoszenia (all-minilm zwraca 384 wymiary -> vector(384))
SELECT AI_GENERATE_EMBEDDINGS(N'Nie pamiętam hasła i nie mogę się zalogować' USE MODEL OllamaEmbeddingi);
```

Ważne uwagi: Ollama domyślnie udostępnia API przez HTTP, a SQL Server wymaga HTTPS – potrzebny jest certyfikat i np. odwrotne proxy. Liczba wymiarów zależy od modelu, więc kolumna musi mieć odpowiedni rozmiar (`vector(384)`, `vector(1536)` itd.). Wektory z różnych modeli nie są ze sobą porównywalne – zmiana modelu oznacza przeliczenie całego archiwum.

Załącznik B. Sprzątanie po warsztacie

Plik / skrypt: cleanup.sql

```
/* Sprzątanie po warsztacie */
USE master;
GO
IF DB_ID(N'WektoryHelpdesk') IS NOT NULL
BEGIN
    ALTER DATABASE WektoryHelpdesk SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE WektoryHelpdesk;
END;
GO
```

Źródła

- Microsoft Learn: Vectors and embeddings in SQL Server – https://learn.microsoft.com/en-us/sql/sql-server/ai/vectors
- Microsoft Learn: CREATE VECTOR INDEX (Transact-SQL) – https://learn.microsoft.com/en-us/sql/t-sql/statements/create-vector-index-transact-sql
- Microsoft Learn: VECTOR_SEARCH (Transact-SQL) – https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-search-transact-sql

- Microsoft Learn: VECTOR_NORMALIZE (Transact-SQL) – https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-normalize-transact-sql
- Microsoft Learn: sys.vector_indexes – https://learn.microsoft.com/en-us/sql/relational-databases/system-catalog-views/sys-vector-indexes-transact-sql
- Microsoft Learn: CREATE EXTERNAL MODEL (Transact-SQL) – https://learn.microsoft.com/en-us/sql/t-sql/statements/create-external-model-transact-sql